
LLM FINE-TUNING:
TEACHING MODELS BEHAVIOR

FINE-TUNING TEACHES BEHAVIOR, NOT FACTS.

Prompt: Tell me how to fine-tune a model

Completion:
How can I control the complexity of a model?
How do I know when my model is done?
How do I test a model?
How do I define a task for my model?
How do I scale a model?
How do I make a probabilistic model?
How do I start using a model once it's complete?
How do I cope with missing data?
How do I make my modeling efforts sustainable?

Prompt: Tell me how to fine-tune a model

Completion:
Fine-tuning a model involves adjusting the parameters of a pre-trained model in order to make it better suited for a given task. There are generally three steps to fine-tuning a model:

1. Select a base model - Select a pre-trained deep learning model that has been trained on a large dataset.
2. Adjust parameters - Adjust parameters of the pre-trained model to better suit the desired task. This may include changing the number of layers, adjusting learning rate, adding regularization, or tweaking the optimizer.
3. Train the model - Train the new model on the desired dataset. The amount of data and the amount of training required will depend on the task and the model.

AGENDA & OVERVIEW

What we will cover

01
Why fine-tune, and when not to

02
Where it sits in post-training

03
Parameter-efficient methods: LoRA and QLoRA

04
Alignment: RLHF, DPO, and reinforcement fine-tuning

05
Quantization and memory math

06
A practical workflow, and fine-tuning vs. RAG

Fine-tuning buys consistency and speed, at the cost of flexibility

PROS

- It handles emotion and slang the way your users write.
- It answers in a brand's tone.
- Style and output format stay consistent.
- It runs faster, because prompts get shorter.
- It is strong on narrow tasks and domains.

CONS

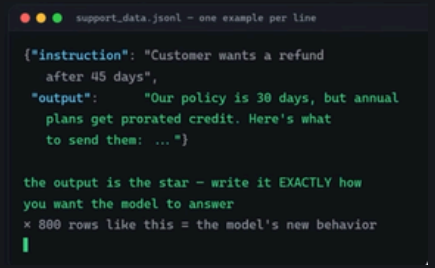
- Retraining is expensive and time-consuming.
- The model is hard to update, and tuning cannot extend its knowledge.
- Catastrophic forgetting can erase earlier skills. That means new training overwrites what the model knew.
- Overfitting makes it memorize examples instead of generalizing.

Fine-Tuning Options

01
Supervised FT

Provide prompt & completions pairs.

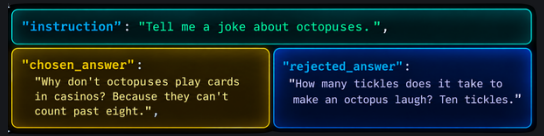
FT model learns how to mimic the style, tone, and structure of those pairs.



02
Preference FT

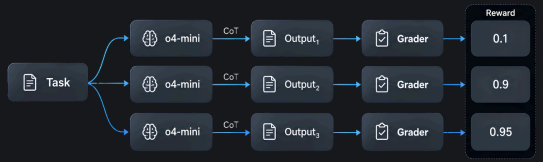
Provide a prompt and preferred & non-preferred response.

FT model learns nuanced preferences from the dataset.



03
Reinforcement FT

RFT uses a grader to distinguish high- and low-quality responses. Trajectories with high grades are reinforced to improve in-domain performance.





PART ONE · CONTEXT

WHERE FINE-TUNING FITS

Post-training turns a base model into a specialist

01

Pre-training
A base model learns from a massive general corpus.

02

Supervised fine-tuning
SFT uses examples to teach instruction following and domain Q&A.

03

Alignment
Preference data (RLHF, DPO) steers toward helpful, safe answers.

04

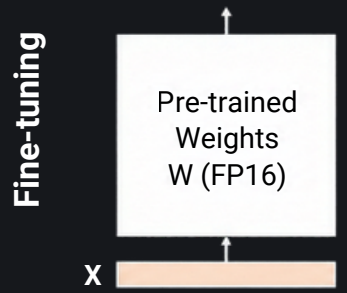
Reasoning
Reward functions on verifiable tasks (RFT, GRPO) build reasoning.

Model names encode this: Gemma 3 4B IT = Gemma 3 family, 4 billion parameters, instruct variant.

SFT Techniques

Full Fine-Tuning

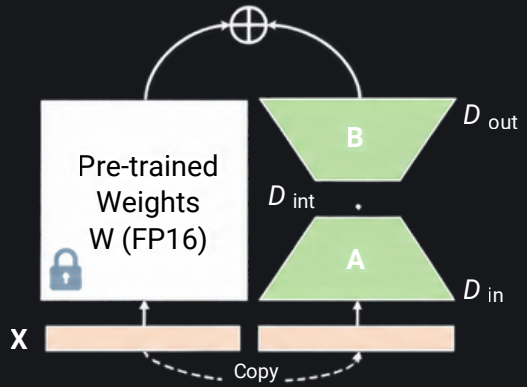
16-bit precision



Best performance
Very high VRAM usage

LoRA

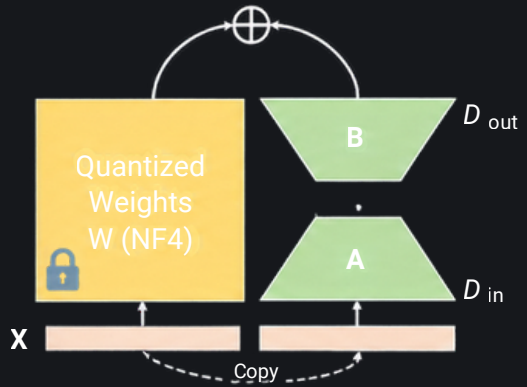
16-bit precision



Quick training
Still costly

QLoRA

4-bit precision



Low RAM usage
Degrades performance

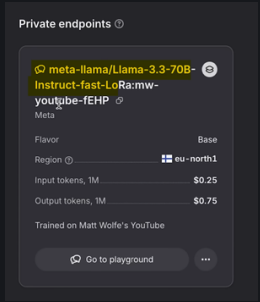
Three ways to access models for fine-tuning

Open-source

You get the weights plus the training code and data.

Open-weight

You retrain the weights on your data. Llama, Mistral, Qwen, DeepSeek, Kimi, and GLM come via Hugging Face or NVIDIA.



Closed-weight

You upload data through an API to OpenAI, Anthropic, Google, or xAI.

Managed services: submit data, the job runs in the backend, and you get a fine-tuned model behind an API.

LIVE DEMO · PICKING THE MODEL

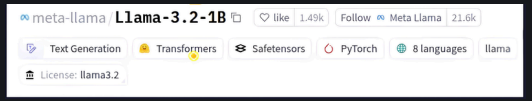
Which model do you actually pick?

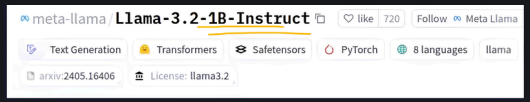
Pick an Instruct model, not Base
Instruct is already trained to follow directions — you're adding your specialty on top.
Base = raw text predictor, much harder starting point

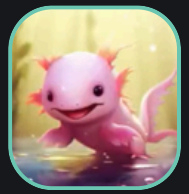
Grab the bnb-4bit variant
e.g. unsloth/Llama-3.1-8B-Instruct-bnb-4bit — weights pre-quantized to 4-bit for QLoRA, ~¼ the download and VRAM of the full model

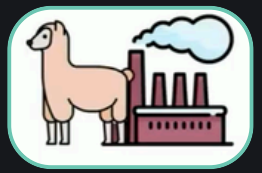
safetensors = trainable weights
the real model — this is what Studio fine-tunes

GGUF = compressed inference format
built for RUNNING models in llama.cpp-style apps. The trainable structure is baked away — you can chat with it, but you can't train it













PART TWO · METHODS

EFFICIENT FINE-TUNING

Full fine-tuning updates every weight, and the bill shows it

VRAM FOR A 10B MODEL

~140 GB

Updating every weightgivesthemost flexibility and the bestquality.VRAMisthe memory on the graphicscard,andfull tuning fills a lot of it.

16-BIT WEIGHT

2 bytes

per parameter

32-BIT WEIGHT

4 bytes

per parameter

ROUGH MEMORY RULE

params × bytes per param + ~20% + KV cache

The KV cache stores attention keys and values while the model runs.

🤗 PEFT

```python
from transformers import AutoModelForSeq2SeqLM
from peft import get_peft_config, get_peft_model, LoraConfig, TaskType
model_name_or_path = "bigscience/mt0-large"
tokenizer_name_or_path = "bigscience/mt0-large"

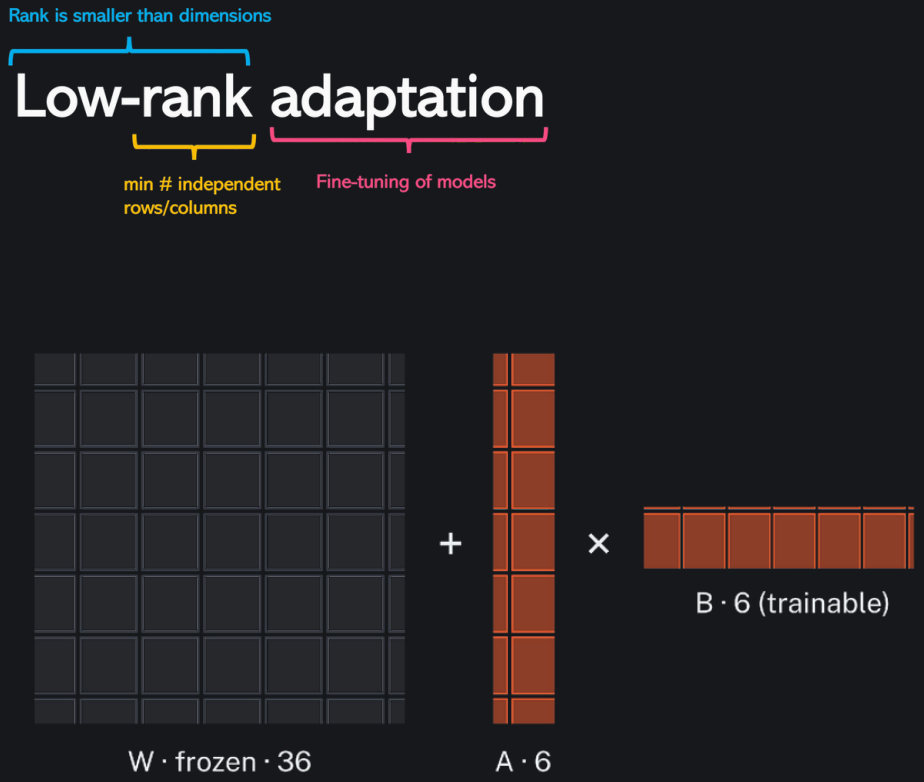
peft_config = LoraConfig(
    task_type=TaskType.SEQ_2_SEQ_LM, inference_mode=False, r=8, lora_alpha=32, lora_dropout=0.1
)

model = AutoModelForSeq2SeqLM.from_pretrained(model_name_or_path)
model = get_peft_model(model, peft_config)
model.print_trainable_parameters()
# output: trainable params: 2359296 || all params: 1231940608 || trainable%: 0.19151053100118282
```

```python
config = {
    "peft_type": "LORA",
    "task_type": "SEQ_2_SEQ_LM",
    "inference_mode": False,
    "r": 8,
    "target_modules": ["q", "v"],
    "lora_alpha": 32,
    "lora_dropout": 0.1,
    "fan_in_fan_out": False,
    "enable_lora": None,
    "bias": "none",
}
```

LoRA freeze the base model, train tiny adapters

- Freeze the pre-trained weights W and learn only the change ΔW.
- Split ΔW into two low-rank matrices, A × B.
- A 6×6 update has 36 values; rank-1 factors need only 12.
- At inference the adapter merges with the frozen weights.
- Higher rank means more capacity and more compute.



QLoRA same adapters, compressed base

01

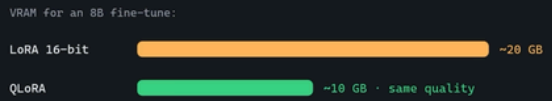
Quantize the base

Frozen base weights drop to 4-bit precision.

02

Add LoRA adapters

Small adapters train on top of the quantized base.



03

Fit one GPU

Large models become tunable on a single GPU, for a small accuracy cost and a large memory saving.

Quantization trades a little accuracy for a lot of memory

PRECISION	MEMORY PER WEIGHT
FP32	full
FP16	half
INT8	¼
4-bit	⅛

Symmetric quantization maps zero to zero. Asymmetric quantization adds a zero-point offset.

Scale = (x_max − x_min) / (q_max − q_min), for example 255 levels for 8-bit.

Post-training quantization calibrates a model that is already trained, with some accuracy loss.

Quantization-aware training learns under quantization, so accuracy holds up better.



PART THREE · ALIGNMENT

ALIGNING MODELS TO PREFERENCES

RLHF, DPO, and RFT align models in different ways

RLHF

SIGNAL
A reward model trained on human preferences guides the LLM through PPO, a reinforcement learning algorithm.
COST
High. Powerful but expensive.
BEST FOR
Broad assistant behavior; it powered ChatGPT.

DPO

SIGNAL
No reward model. It optimizes directly on preferred versus rejected answer pairs.
COST
Lower compute.
BEST FOR
The default for most open-model teams.

RFT

SIGNAL
The model generates many answers to math or code tasks, and correct ones earn a reward.
COST
Needs verifiable tasks.
BEST FOR
Reasoning, as behind OpenAI o-series and DeepSeek-R1.

A practical fine-tuning workflow

PHASE 1 · PREPARATION & SELECTION

01
Define the objective and a success metric.

02
Optimize the prompt and context first.

03
Gather and label data. Quality beats quantity.

04
Choose a transformer base model sized to the task.

PHASE 2 · EXECUTION & EVALUATION

05
Train with LoRA or QLoRA
learning rate ~1e-5 to 1e-6, batch 16–32

06
Evaluate with task-specific metrics for classification, sentiment, or QA.

07
Escalate only if needed: full fine-tuning, then DPO, RLHF, or RFT.

Fine-tuning or RAG? It depends on how often knowledge changes

RAG

Knowledge that moves

Retrieval-augmented generation looks up documents at query time. Use it when data updates frequently, because it extends what the model knows.

FINE-TUNING

Behavior that stays

Use it for stable data updated quarterly or yearly. It bakes in style and behavior, and it cuts latency for conversational AI by avoiding vector lookups.

Often best combined: fine-tune for behavior, and use RAG for facts.

Myths, and why fine-tuning still matters

COMMON MISCONCEPTIONS

- ~~It teaches the model facts.~~ It shapes behavior.
- ~~It needs huge datasets~~. Small, clean data often works.
- ~~It is always expensive and complicated~~. LoRA and QLoRA lower the bar.

WHERE IT STILL WINS

- Low latency and cheaper inference than reasoning models.
- Distillation, where small models learn from large ones.
- RFT for reasoning. A tuned small model can beat a large base model.

Try first: skills, context engineering, and better prompts.